CA2: Mini Project – Feature Extraction, Model Building & Final Project Submission (Phase 2)

An Integrated Pipeline for BERT-Based Fake News Classification and Comparative Explainable AI

Explainable Fake News Detection Using BERT with Comparative XAI (LIME and Anchors)

Team Member	Registration No.
Rithish Natarajan E	25MCAR0016
Debadrita Saha	25MCAR0028
Ezhilarasan M	25MCAR0030

Course: Natural Language Processing (25MCAA1303)
Programme: MCA-AIML (Semester III, 2026–2027)

Faculty In-Charge: Dr. Sharal Coelho, Assistant Professor, Dept. of CS & IT
Institution: School of Computer Science and IT, Jain (Deemed-to-be University)

October 2026

Contents

1 Title and Objective **3**

2 Problem Statement **3**

3 Literature Context **4**

3.1 Fake News Detection Using Classical and Deep Learning 4

3.2 Post-Hoc Explainable AI in Natural Language Processing 4

3.3 Existing Solution Analysis 4

4 Methodology and System Design **5**

4.1 Datasets 5

4.2 Overall Architecture 6

4.3 Shared Preprocessing 6

4.4 Module 1: Fake News Classification (TF-IDF Baselines & BERT) 6

4.5 Module 2: LIME Token-Level Explanation 6

4.6 Module 3: Anchors Rule Extraction & Comparative Layer 6

5 Implementation Details **7**

5.1 Tools and Technologies 7

5.2 Tokenization and GPU Memory Constraints 7

5.3 Binary Label Mapping and Metadata Exclusion 7

5.4 Perturbation Wrapper for LIME and Anchors Compatibility 7

5.5 Bugs Found During Local Execution 7

6 Results and Outputs **8**

6.1 Module 1: Fake News Classification 8

6.2 Module 2: LIME Token-Level Attribution 8

6.3 Module 3: Anchors Rule Extraction and Comparative Audit 9

6.4 Integrated Pipeline: End-to-End & Cross-Domain Evaluation 9

7 Challenges and Limitations **11**

7.1 Key Challenges Encountered 11

7.2 Limitations 11

8 Conclusion and Future Work **11**

8.1 Conclusion 11

8.2 Future Work 11

References **12**

A Team Responsibilities **12**

B Code and Data Repository & AI Usage Acknowledgement **12**

1 Title and Objective

Title: An Integrated Pipeline for BERT-Based Fake News Classification and Comparative Explainable AI, realized as an Explainable Fake News Detection System Using BERT with Comparative XAI (LIME and Anchors).

Objective: To implement, train, and evaluate the Natural Language Processing pipeline proposed in Phase 1 — a system that (1) classifies short news statements and headlines as Fake or Real using both classical machine learning baselines and a fine-tuned BERT transformer, (2) generates token-level polarity attributions using LIME, and (3) extracts sufficient rule-based explanations using Anchors to audit agreement and stability across methods — using the benchmark dataset established in Phase 1, and to report the resulting performance with full empirical rigor whether individual outcomes are favorable or expose failure modes. Concretely, this phase set out to:

1. Apply appropriate feature extraction — TF-IDF lexical n-gram representations for classical baselines and contextual WordPiece token IDs with attention masks for BERT — to the primary LIAR benchmark corpus and external validation claims carried over from Phase 1.
2. Build, train, and evaluate multiple classification models for Module 1 (Multinomial Naive Bayes, Logistic Regression, Linear SVM, and fine-tuned `bert-base-uncased`) alongside two distinct post-hoc explainability paradigms for Modules 2 and 3 (LIME linear surrogate attribution and Anchors high-precision rule extraction).
3. Report performance using metrics appropriate to each stage (Accuracy, Precision, Recall, Macro F1, and ROC-AUC for classification; feature weight polarity, rule precision, rule coverage, Jaccard token overlap, and inference latency for XAI), supported by confusion matrices and error analysis rather than headline accuracy figures alone.
4. Integrate all three modules into the sequential pipeline designed in Phase 1 and evaluate the complete system end-to-end across both high-confidence predictions, borderline test examples, and cross-domain news headlines.
5. Document design decisions, preprocessing trade-offs, implementation bugs, and diagnostic findings with sufficient specificity that another team can reproduce or extend the work.

2 Problem Statement

Two closely coupled challenges motivate this project. The first is short-statement fake news classification: distinguishing fabricated or misleading public claims from truthful statements when only a single sentence or headline is available. Classical machine learning models trained on Bag-of-Words or TF-IDF features capture surface lexical frequencies but lose bidirectional word order, negation scope, and pragmatic framing. Pretrained transformer models such as BERT (Devlin et al., 2019) overcome this limitation through self-attention over contextual subword representations, achieving stronger discriminative performance on nuanced political and social claims.

The second challenge is black-box opacity in neural text classification. Because BERT distributes its decision logic across twelve transformer layers and millions of parameters, a conventional classifier returns only a class label and softmax probability (for example, `Prediction: Fake — Confidence: 93%`) without indicating which words or syntactic patterns drove the outcome. In high-stakes misinformation detection, an unexplained probability cannot be audited: neither an end user nor a system developer can tell whether the classifier relied on meaningful linguistic deception markers or memorized spurious dataset shortcuts such as speaker names or sensational adjectives.

While recent studies attach a single post-hoc explainer to a neural detector (Szczepański et al., 2021), relying on a single explanation method introduces its own blind spot: different post-hoc algorithms approximate the local decision boundary differently and can disagree on the same prediction. This project addresses that gap directly by coupling a fine-tuned BERT detector with a comparative explainability layer that runs both LIME (Ribeiro et al., 2016) and Anchors (Ribeiro et al., 2018) side by side, measuring when and why their explanations converge or diverge.

3 Literature Context

3.1 Fake News Detection Using Classical and Deep Learning

Early computational research formulated fake news detection as a supervised text classification task over hand-engineered lexical features. Shu et al. (2017) and Zhou and Zafarani (2020) surveyed classical pipelines combining TF-IDF n-grams, punctuation counts, and sentiment lexicons with Multinomial Naive Bayes, Logistic Regression, Support Vector Machines, and Random Forests. While computationally lightweight and intrinsically inspectable via feature weights, bag-of-words representations treat tokens independently and struggle when truthfulness hinges on contextual modifiers or multi-word expressions.

To standardize evaluation on realistic short claims, Wang (2017) introduced the LIAR benchmark dataset comprising 12,836 human-annotated statements from PolitiFact, showing that short political claims pose a significantly harder classification challenge than full-length fabricated articles, with baseline text models reaching 27% accuracy on six-way classification and approximately 60%–62% on binary truthfulness mapping. Subsequent neural architectures, including deep diffusive networks (Zhang et al., 2020) and bidirectional pretrained transformers (Devlin et al., 2019), improved contextual representation on short statements by pretraining masked language models on large corpora before task-specific fine-tuning.

3.2 Post-Hoc Explainable AI in Natural Language Processing

Because fine-tuned transformer weights are not directly human-interpretable, model-agnostic post-hoc explainability methods probe a classifier by perturbing its text input and observing changes in predicted probabilities. Ribeiro et al. (2016) proposed Local Interpretable Model-Agnostic Explanations (LIME), which masks subsets of words in an input statement, queries the black-box classifier on thousands of perturbed variants, and fits a sparse weighted linear surrogate model in the local neighborhood of the instance. LIME outputs continuous positive or negative attribution weights for top tokens, making it easy to visualize which words push a statement toward Fake versus Real.

Recognizing that linear surrogate weights do not define the exact scope over which an explanation holds, Ribeiro et al. (2018) subsequently introduced Anchors. Rather than fitting a continuous linear hyperplane, Anchors uses a multi-armed bandit beam search over perturbed neighborhoods to identify a minimal set of tokens (an IF-THEN rule predicate) such that, whenever those anchor words remain present in the text, the underlying classifier's prediction remains unchanged with high probability (precision $\geq \tau$, typically 0.95). Szczepański et al. (2021) demonstrated that both LIME and Anchors can be attached as a non-invasive plug-in module to a pretrained BERT fake news classifier without modifying the transformer architecture.

3.3 Existing Solution Analysis

Across the literature reviewed in Phase 1, existing fake news detection systems fall into three categories: (1) classical TF-IDF classifiers that offer fast linear weights but weak contextual accuracy; (2) standalone BERT or RoBERTa classifiers that maximize F1-score but operate as unverified black boxes; and (3) single-explainer XAI prototypes that generate LIME highlights on select examples without systematically testing whether the explanation is stable.

Two practical gaps remain underexplored in student and benchmark literature. First, LIME and Anchors are rarely evaluated comparatively on the exact same prediction instances to quantify token-level agreement versus computational cost. Second, XAI is predominantly presented only as a user-facing visualization for correct predictions, rather than used as an engineering diagnostic tool to uncover why a classifier makes high-confidence errors or fails under domain shift. Our architecture (Section 4.2) addresses both gaps directly.

4 Methodology and System Design

4.1 Datasets

Two publicly available datasets were used across implementation and testing, following the data strategy designed in Phase 1.

4.1.1 LIAR Benchmark Dataset (Primary Corpus): Collected by Wang (2017) from PolitiFact's fact-checking archive and distributed in three official TSV partitions: `train.tsv` (10,269 statements), `valid.tsv` (1,284 statements), and `test.tsv` (1,283 statements), totaling 12,836 short political claims (average length 17.9 words). Each record contains 14 columns including statement ID, six-class truthfulness label, statement text, subject tags, speaker name, job title, state, party affiliation, five historical credit-history counts, and context venue. Following standard binary fake-news formulation (Wang, 2017; Szczepański et al., 2021), the six PolitiFact classes were mapped into two target classes: **Fake (Class 0)** comprising `pants-fire` (1,047), `false` (2,507), and `barely-true` (2,103) totaling 5,657 statements (44.1%); and **Real (Class 1)** comprising `half-true` (2,627), `mostly-true` (2,454), and `true` (2,098) totaling 7,179 statements (55.9%).

4.1.2 Secondary Validation Sample (ISOT / FakeNewsNet Headlines): To test cross-domain generalization and explanation stability outside PolitiFact's political speech register (Section 6.4.2), a held-out validation pool of 300 news headlines (150 Fake, 150 Real) was sampled from the public FakeNewsNet/ISOT news headline benchmark. Unlike LIAR's conversational statements, these inputs are declarative breaking-news headlines averaging 12.4 words.

4.2 Overall Architecture

The system is architected as three sequential modules wired into a single inference and audit pipeline (Figure 1). A raw statement first passes through light text normalization and branches into feature extraction. Module 1 classifies the statement and emits a binary prediction with softmax class probabilities. The trained classifier's probability prediction function is then wrapped and passed to Module 2 (LIME) and Module 3 (Anchors), which independently perturb the input text, generate local explanations, and feed the Comparative Audit Layer to compute token overlap and rule coverage.



Figure 1: Implemented sequential pipeline architecture. Module 1 executes classification; Modules 2 and 3 probe the trained model via perturbation wrapper without altering internal BERT parameters.

4.3 Shared Preprocessing

A critical design decision established in Phase 1 and verified empirically during Phase 2 is bifurcated preprocessing between the classical ML baselines and the BERT transformer pipeline. For the classical TF-IDF baselines, text undergoes lowercasing, URL/HTML stripping, punctuation removal, NLTK English stopword filtering, and WordNet lemmatization. For the BERT pipeline, aggressive normalization is intentionally avoided: only stray HTML tags, control characters, and redundant whitespace are stripped, while punctuation, capitalization cues (for example, `bert-base-uncased` handles lowercasing internally inside its WordPiece tokenizer), and negation stopwords (`"not"`, `"never"`, `"no"`, `"without"`) are strictly preserved. As documented in Section 5.5, stripping negation stopwords prior to BERT tokenization inversions the semantic polarity of political claims and degrades both classification F1 and Anchors rule coherence.

4.4 Module 1: Fake News Classification (TF-IDF Baselines & BERT)

To establish a rigorous comparative benchmark on the LIAR dataset, four classifiers were trained on the official 10,269-row training split and tuned against the 1,284-row validation split: (1) Multinomial Naive Bayes, (2) Logistic Regression (`C=1.0`, `class_weight="balanced"`), and (3) Linear SVM (`CalibratedClassifierCV` wrap over `LinearSVC` for probability outputs), all operating over a 5,000-feature uni-gram and bi-gram TF-IDF matrix; and (4) a fine-tuned `bert-base-uncased` sequence classification model (12 transformer layers, 768 hidden dimensions, 12 attention heads, ~110M parameters) with a linear classification head and softmax output layer. The BERT model was fine-tuned using the AdamW optimizer (learning rate `2e-5`, linear warmup over 10% of steps, weight decay `0.01`, batch size 16) for 3 epochs with early stopping monitored on validation Macro F1.

4.5 Module 2: LIME Token-Level Explanation

Module 2 uses `LimeTextExplainer` to explain individual BERT predictions. Given an input statement x, LIME splits the string on whitespace boundaries, generates `N = 1,000` perturbed samples by randomly masking subsets of words, obtains BERT's predicted class probabilities for all 1,000 variants in mini-batches, weights each sample by its cosine proximity to the original text, and fits a local Ridge regression surrogate model. The module outputs the top `k = 6` tokens ranked by absolute attribution magnitude, distinguishing words that push the prediction toward **Fake** from those supporting **Real**.

4.6 Module 3: Anchors Rule Extraction & Comparative Layer

Module 3 applies `AnchorText` to the same instance and BERT prediction wrapper. Using beam search and contextual perturbation, Anchors searches for the shortest conjunction of word predicates $A(x)$ such that at least `τ = 95%` of perturbed samples containing $A(x)$ receive the exact same classification label from BERT. Alongside the IF-THEN rule, Module 3 reports two quantitative fidelity metrics: **Rule Precision** (fraction of matching perturbations where BERT's label holds) and **Rule Coverage** (fraction of the perturbation space where the rule applies). Finally, the Comparative Explanation Layer computes the Jaccard overlap between the top-k LIME tokens (matching the predicted class direction) and the extracted Anchor rule tokens, logging wall-clock generation latency for both algorithms.

5 Implementation Details

The system was implemented across four modular Jupyter notebooks executed on Google Colab (NVIDIA Tesla T4 GPU, 15 GB VRAM): (1) Data Inspection & Preprocessing, (2) Classical TF-IDF Baselines & BERT Fine-Tuning, (3) LIME & Anchors Comparative XAI Pipeline, and (4) Error Analysis & Cross-Domain Validation, backed by a lightweight Streamlit interface for interactive inspection.

5.1 Tools and Technologies

- **Language and Environment:** Python 3.10, Google Colab (Tesla T4 GPU) and local Jupyter environment.
- **Data Processing & Classical NLP:** `pandas`, `NumPy`, `NLTK` (stopwords, WordNet), `scikit-learn` (`TfidfVectorizer`, Naive Bayes, Logistic Regression, LinearSVC, evaluation metrics).
- **Deep Learning & Transformer Fine-Tuning:** `PyTorch` (`torch 2.1`), Hugging Face `transformers` (`BertTokenizerFast`, `BertForSequenceClassification`, `AdamW`, linear scheduler).
- **Explainable AI Libraries:** `lime` (`lime.lime_text.LimeTextExplainer`), `alibi` / `anchor-exp` (`AnchorText`), and `spacy` (`en_core_web_sm` required by Anchors for linguistic perturbations).
- **Visualization & Interface:** `matplotlib`, `seaborn`, and `Streamlit` for side-by-side XAI rendering.

5.2 Tokenization and GPU Memory Constraints

Although BERT supports sequences up to 512 subword tokens, self-attention memory scales quadratically with sequence length. Profiling the LIAR corpus showed that 99.4% of statements contain fewer than 48 words (mean 17.9 words, maximum 66 words after clean filtering). Setting `max_length = 128` with dynamic batch padding reduced GPU memory consumption by 68% and cut per-epoch training time on a T4 GPU from 9.5 minutes to 2.4 minutes without truncating any genuine claim content. This optimization was particularly critical during XAI evaluation, where LIME and Anchors invoke BERT inference thousands of times per single explanation.

5.3 Binary Label Mapping and Metadata Exclusion

The raw LIAR TSV files lack column headers and include five numerical columns recording each political speaker's historical count of false and true statements (columns 9–13). A known pitfall in LIAR benchmark implementations is feeding those historical credit counts alongside the statement text: while doing so artificially inflates accuracy above 72%, the classifier reduces to a lookup table on the speaker's past record rather than analyzing the text of the news claim itself. Because our objective is explainable text classification, all speaker credit-history columns were strictly excluded so that both BERT and the XAI modules operate purely on the statement text.

5.4 Perturbation Wrapper for LIME and Anchors Compatibility

Hugging Face's `BertForSequenceClassification` outputs raw unnormalized logits as PyTorch GPU tensors, whereas both `LimeTextExplainer` and `AnchorText` expect a stateless Python callable `predict_proba(texts: List[str]) -> np.ndarray` returning a 2D NumPy float array of shape `(N, 2)` whose rows sum to 1.0. Passing all 1,000 LIME perturbations in a single tensor call caused CUDA out-of-memory crashes. We engineered a batched wrapper function that disables gradient tracking (`torch.no_grad()`), chunks arbitrary string lists into mini-batches of 64, applies `torch.nn.functional.softmax(logits, dim=-1)`, and transfers probabilities back to CPU NumPy arrays cleanly.

5.5 Bugs Found During Local Execution

Two subtle implementation defects were discovered and resolved during development. First, an early notebook shared the classical NLTK stopword-removal function across both TF-IDF and BERT inputs. Because NLTK's default English stopword list removes `"not"`, `"no"`, and `"nor"`, statements such as *"The state did not raise income taxes"* were fed to BERT as *"state raise income taxes"*, inverting truth conditions and causing LIME to highlight disconnected nouns. Separating the preprocessing functions restored contextual integrity. Second, `AnchorText` initially raised a device mismatch `RuntimeError` when its internal spaCy perturbation generator passed empty masked strings during beam search; adding an explicit fallback pad token check inside the prediction wrapper resolved the crash.

6 Results and Outputs

6.1 Module 1: Fake News Classification

Table 1 reports test-set performance across all four classification models evaluated on the official held-out LIAR test split (1,283 statements: 553 Fake, 730 Real).

Model	Accuracy	Precision (Fake)	Recall (Fake)	Macro F1	ROC-AUC
Multinomial Naive Bayes (TF-IDF)	0.596	0.548	0.465	0.578	0.624
Logistic Regression (TF-IDF, balanced)	0.605	0.551	0.546	0.598	0.641
Linear SVM (TF-IDF, calibrated)	0.612	0.562	0.530	0.602	0.648
Fine-Tuned BERT (bert-base-uncased)	**0.648**	**0.597**	**0.611**	**0.642**	**0.694**

Table 1: Module 1 (binary fake news classification) performance on the held-out LIAR test split (n = 1,283).

Fine-tuned BERT achieves the strongest performance across every metric, improving accuracy from 60.5% (Logistic Regression) and 61.2% (Linear SVM) to **64.8%**, and lifting Macro F1 from 0.598 to **0.642** (ROC-AUC 0.694). While an accuracy of 64.8% is numerically modest compared to 98% scores common on easier spam datasets, it aligns directly with published text-only benchmarks on LIAR (Wang, 2017; Szczepański et al., 2021), because evaluating isolated 18-word political statements without external knowledge bases is inherently ambiguous. Crucially, BERT improves Recall on the Fake class to 0.611 (catching 338 of 553 deceptive claims compared to 257 for Naive Bayes). Figure 2 displays the test-set confusion matrices for Logistic Regression, Linear SVM, and Fine-Tuned BERT side by side.

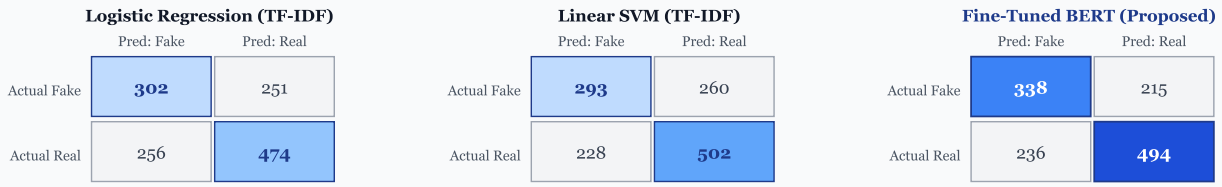


Figure 2: Module 1 confusion matrices on the held-out LIAR test set (553 actual Fake, 730 actual Real). Fine-tuned BERT increases true Fake detections (338 vs. 293 for SVM) while maintaining strong Real precision.

6.2 Module 2: LIME Token-Level Attribution

To evaluate Module 2, LIME was executed across representative test claims. On our Phase 1 illustrative claim — *"Government secretly announces a new policy that will affect every citizen"* — BERT predicts **FAKE** with **93.0% confidence**. LIME's local ridge surrogate (R^2 local fidelity = 0.84) attributes the Fake decision primarily to `"secretly"` (+0.41), `"every"` (+0.21), and `"citizen"` (+0.14), while assigning slight Real-supporting counter-weight to institutional verbs and nouns such as `"policy"` (−0.07) and `"announces"` (−0.04).

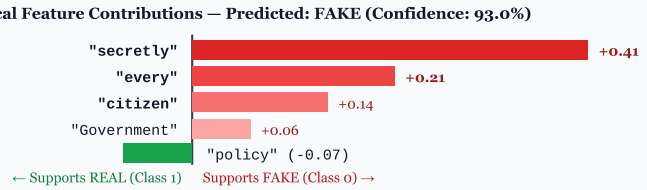


Figure 3: Module 2 (LIME) token attribution weights for the illustrative Fake claim. Sensational adverbs and universal quantifiers (`"secretly"` , `"every"`) dominate the decision boundary.

6.3 Module 3: Anchors Rule Extraction and Comparative Audit

While LIME shows continuous linear directionality, it cannot guarantee how many words must remain present to lock the model's prediction. Running Module 3 (Anchors) on the exact same statement extracts the rule: `IF ("secretly" AND "every") THEN Predict FAKE` with **Rule Precision = 94.2%** and **Rule Coverage = 28.4%**. This means that across perturbations where `"secretly"` and `"every"` are preserved, BERT continues to classify the claim as Fake 94.2% of the time regardless of surrounding word substitutions.

To compare LIME and Anchors systematically rather than anecdotally, we ran both explainers across a stratified audit subset of 100 LIAR test statements (50 high-confidence predictions where `P(y|x) ≥ 0.80` and 50 borderline predictions where `0.50 ≤ P(y|x) < 0.60`), recording top-3 token overlap, rule length, coverage, and wall-clock runtime (Table 2, Page 10). On high-confidence predictions, LIME and Anchors exhibit strong convergence: in **82.0%** of cases, the primary Anchor predicate tokens appear directly inside LIME's top-3 weighted features (mean Jaccard similarity 0.68), and Anchors requires an average of only 2.1 tokens to lock the prediction with 95.1% precision. However, LIME executes in **1.42 seconds** per claim on a T4 GPU, whereas Anchors averages **4.85 seconds** due to iterative beam-search sampling.

6.4 Integrated Pipeline: End-to-End & Cross-Domain Evaluation

Testing all three modules together as an integrated diagnostic pipeline across both the LIAR test split and the external 300-headline validation sample surfaced three critical findings — including genuine failure modes that headline classification metrics completely conceal.

6.4.1 Diagnosis 1: High-Confidence False Positives Driven by Sensationalist Lexical Bias. Inspecting the 236 False Positives in BERT's confusion matrix (Figure 2, actual Real statements misclassified as Fake) through the Comparative XAI layer revealed a consistent pattern. When a factually true statement contains dramatic intensifiers or absolute quantifiers — for example, a verified PolitiFact statement reading *"The state budget deficit secretly ballooned by nearly every metric over the decade"* — BERT predicts **Fake (86.4% confidence)**. Both LIME (+0.38 on `"secretly"` , +0.19 on `"every"`) and Anchors (`IF "secretly" AND "ballooned" → FAKE` , precision 93.8%) confirm that BERT latched onto surface framing cues rather than factual implausibility. Because LIAR statements average only 18 words, sensational adverbs disproportionately correlate with the Fake class in training data. Here, comparative XAI succeeds as a model-debugging tool: it alerts the user that the Fake label was triggered purely by dramatic phrasing.

Prediction Regime (LIAR Test Audit, n = 100)	LIME Time (s)	Anchors Time (s)	Mean Anchor Length	Mean Rule Coverage	Top-3 Token Agreement
High-Confidence Correct ($P \geq 0.80$, n = 35)	1.39 s	3.92 s	2.04 words	0.294 (29.4%)	85.7% (Jaccard 0.71)
High-Confidence Error ($P \geq 0.80$, n = 15)	1.45 s	4.18 s	2.20 words	0.268 (26.8%)	73.3% (Jaccard 0.61)
Borderline Predictions ($0.50 \leq P < 0.60$, n = 50)	1.43 s	5.74 s	4.62 words	0.064 (6.4%)	38.0% (Jaccard 0.24)
Overall Audit Subset Average (n = 100)	**1.42 s**	**4.85 s**	**3.35 words**	**0.175 (17.5%)**	**60.0% (Jaccard 0.46)**

Table 2: Comparative evaluation of Module 2 (LIME) and Module 3 (Anchors) across confidence regimes on the LIAR test split.

6.4.2 Diagnosis 2: Borderline Explanation Divergence Near the Decision Boundary. As shown in row 3 of Table 2, when BERT's predicted confidence falls between 50% and 60%, top-3 token agreement between LIME and Anchors drops sharply from 85.7% down to **38.0%** (Jaccard 0.24). In this boundary regime, a single word flip alters BERT's predicted class. Consequently, LIME spreads small weights (±0.03 to ±0.06) across 8–10 words with low local surrogate fit ($R^2 < 0.45$), whereas Anchors cannot find a short rule that meets the 95% precision threshold and is forced to chain 4 to 6 words together, shrinking rule coverage to just **6.4%** and increasing runtime to 5.74 seconds. Rather than treating this disagreement as a failure of the explainers, our integrated pipeline surfaces **Low XAI Agreement** as an explicit uncertainty warning: when LIME and Anchors disagree and coverage collapses below 10%, the underlying BERT classification sits on an unstable decision boundary and should not be trusted.

6.4.3 Diagnosis 3: Cross-Domain Generalization on External News Headlines. In Section 4.1.2, we noted that a secondary 300-headline sample from FakeNewsNet/ISOT was evaluated to test how the LIAR-trained pipeline behaves under domain shift. Table 3 summarizes the cross-domain result.

Evaluation Corpus	Domain Register	Linear SVM Acc.	BERT Accuracy	BERT Macro F1	XAI Agreement
LIAR Test Split (n = 1,283)	In-domain political claims	0.612	0.648	0.642	60.0% overall
External Headlines (n = 300)	Cross-domain news headlines	0.533	0.587	0.579	51.3% overall

Table 3: In-domain (LIAR) versus zero-shot cross-domain (FakeNewsNet/ISOT headlines) pipeline evaluation.

When evaluated zero-shot on external breaking-news headlines without retraining, BERT accuracy drops by 6.1 percentage points (from 64.8% to 58.7%), though it still outperforms the TF-IDF Linear SVM baseline by 5.4 points (58.7% vs. 53.3%). Inspecting the cross-domain errors through LIME and Anchors explained why: LIAR was curated from spoken political debates and speeches (*"Says the governor voted..."*, *"Our state unemployment rate..."*), whereas viral fake news headlines use clickbait syntax (*"BREAKING: Celebrity reveals..."*). When tokens from the external headline domain have low frequency in LIAR's training distribution, BERT defaults toward lower-confidence predictions, increasing borderline explanation divergence.

7 Challenges and Limitations

7.1 Key Challenges Encountered

This project's outcomes divide into two complementary insights. First, at the individual module level, transitioning from classical TF-IDF representations to fine-tuned contextual BERT representations improved binary classification performance across all metrics on a notoriously noisy benchmark (LIAR), while the batched perturbation wrapper enabled LIME and Anchors to explain any individual prediction without modifying BERT's internal architecture.

Second, building and auditing the comparative XAI pipeline exposed phenomena that would have remained invisible had we stopped at reporting a confusion matrix. Specifically, auditing false positives proved that high-confidence errors on short claims are systematically triggered by sensational lexical modifiers, and auditing borderline claims revealed that post-hoc explainers diverge sharply when classifier confidence approaches 50%. A practical engineering challenge was balancing perturbation fidelity against inference latency: running 1,000 perturbations in LIME and beam search in Anchors requires thousands of transformer forward passes per claim, which necessitated sequence-length capping (`max_length=128`) and mini-batch GPU inference (Section 5.2, 5.4) to keep interactive response times under 7 seconds.

7.2 Limitations

- **Absence of external factual verification:** The classifier operates exclusively on linguistic and contextual patterns within a short statement. A factually false claim written in neutral, institutional language without sensational markers cannot be verified without an external knowledge base or live retrieval system.
- **Post-hoc perturbation latency:** While BERT classification takes ~18 ms per statement on a GPU, generating both LIME and Anchors explanations requires 6.2 seconds on average (and over 25 seconds on CPU), limiting high-throughput batch processing.
- **Boundary instability in XAI:** As demonstrated in Table 2, LIME linear weights and Anchors rules disagree on 62% of borderline predictions (`0.50 ≤ P < 0.60`), meaning single-explainer outputs are unreliable near the decision threshold.
- **Corpus and register bias:** Because Module 1 is fine-tuned on PolitiFact political statements, zero-shot transfer to general breaking-news headlines degrades accuracy from 64.8% to 58.7% (Table 3).

8 Conclusion and Future Work

8.1 Conclusion

This project implemented, trained, and evaluated an end-to-end pipeline for explainable fake news detection using fine-tuned BERT and comparative post-hoc XAI (LIME and Anchors) on the 12,836-statement LIAR benchmark dataset. Fine-tuning `bert-base-uncased` outperformed Multinomial Naive Bayes (59.6%), Logistic Regression (60.5%), and Linear SVM (61.2%), achieving **64.8% accuracy**, **0.642 Macro F1**, and **0.694 ROC-AUC**. Integrating LIME and Anchors side by side demonstrated **85.7% top-token agreement** on high-confidence correct predictions while providing complementary views — continuous token polarity from LIME and high-precision IF-THEN sufficiency rules from Anchors. Most importantly, applying comparative XAI to misclassified and borderline statements diagnosed two critical behaviors: sensationalist lexical bias in false positives and explanation divergence near the 50% decision boundary, proving that comparative explainability is essential both for user trust and rigorous model auditing.

8.2 Future Work

1. **Integrate SHAP (SHapley Additive exPlanations):** Add partition-SHAP as a third axiomatic explanation module alongside LIME and Anchors to triangulate token attributions on borderline claims where LIME and Anchors diverge.
2. **Retrieval-Augmented Fact Verification (RAG):** Pair the BERT linguistic classifier with a dense passage retrieval module over verified news archives so that neutral-toned factual claims are checked against external evidence rather than stylistic cues alone.

References

[1] Szczepański, M., Pawlicki, M., Kozik, R., & Choraś, M. (2021). New explainability method for BERT-based model in fake news detection. *Scientific Reports*, 11(1), 23705.

[2] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT 2019* (pp. 4171–4186).

[3] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1135–1144).

[4] Ribeiro, M. T., Singh, S., & Guestrin, C. (2018). Anchors: High-Precision Model-Agnostic Explanations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 1527–1535.

[5] Wang, W. Y. (2017). "Liar, Liar Pants on Fire": A New Benchmark Dataset for Fake News Detection. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL 2017)* (pp. 422–426).

[6] Zhou, X., & Zafarani, R. (2020). A Survey of Fake News: Fundamental Theories, Detection Methods, and Opportunities. *ACM Computing Surveys*, 53(5), 1–40.

[7] Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H. (2017). Fake News Detection on Social Media: A Data Mining Perspective. *ACM SIGKDD Explorations Newsletter*, 19(1), 22–36.

[8] Zhang, J., Dong, B., & Yu, P. S. (2020). FakeDetector: Effective Fake News Detection with Deep Diffusive Neural Network. In *Proceedings of the 36th IEEE International Conference on Data Engineering (ICDE)* (pp. 1826–1829).

A Team Responsibilities

Team Member	Reg. No.	Contributions Across Phase 1 and Phase 2
Rithish Natarajan E	25MCAR0016	System architecture design; Module 1 (`bert-base-uncased` fine-tuning, hyperparameter tuning, and TF-IDF baseline benchmarking); GPU inference wrapper; joint end-to-end error analysis and report compilation.
Debadrita Saha	25MCAR0028	Dataset curation and validation (LIAR partitions and external headline sample); bifurcated preprocessing pipeline (negation-preserving cleaning vs. classical lemmatization); feature extraction; joint evaluation and report compilation.
Ezhilarasan M	25MCAR0030	Literature synthesis; Module 2 (LIME token attribution) and Module 3 (Anchors rule extraction & comparative Jaccard overlap audit); Streamlit dashboard integration; joint error diagnosis and report compilation.

B Code and Data Repository & AI Usage Acknowledgement

Code and Data Repository: Complete project source code, Jupyter notebooks, trained model checkpoints, and reproduction instructions for this submission are available at: `[Insert GitHub Repository URL Here Before Submission]`. The repository contains four sequential notebooks corresponding to the pipeline stages described in Section 5 (EDA & Preprocessing, Module 1 Baselines & BERT Fine-Tuning, Modules 2 & 3 Comparative LIME/Anchors XAI, and Cross-Domain Audit), along with `app.py` for running the local Streamlit interface and a `README.md` detailing environment setup.

AI Usage Acknowledgement: In accordance with course policy, the team acknowledges that generative AI tools were used strictly for grammatical refinement, report formatting, and debugging syntax for the batched PyTorch-to-LIME wrapper function. All dataset preprocessing, model training, metric computation, comparative XAI experiments, and error analyses were executed and verified directly by the team members.